# 数据开发

2026年4月1日 18:35

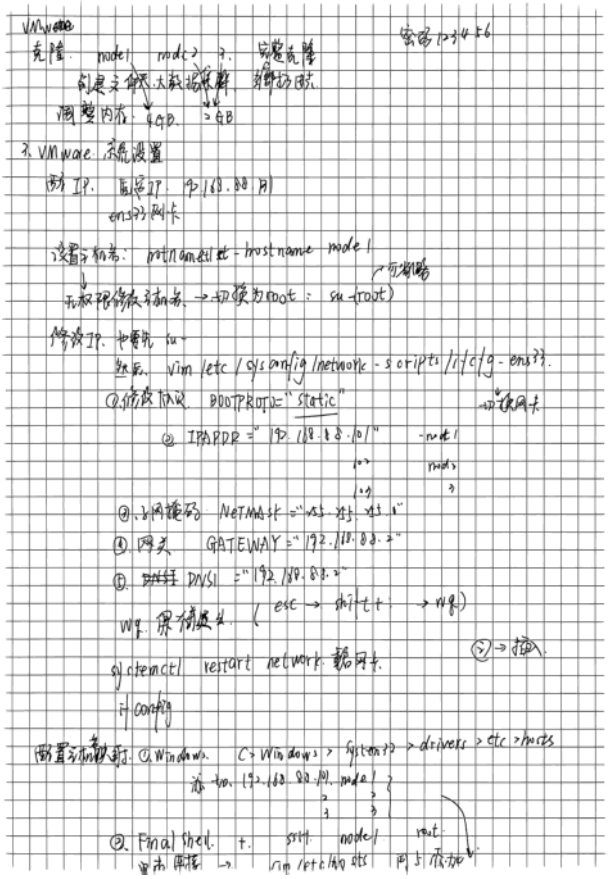
写在开头：不要在下载最新版本了！一定要考虑兼容问题！！！

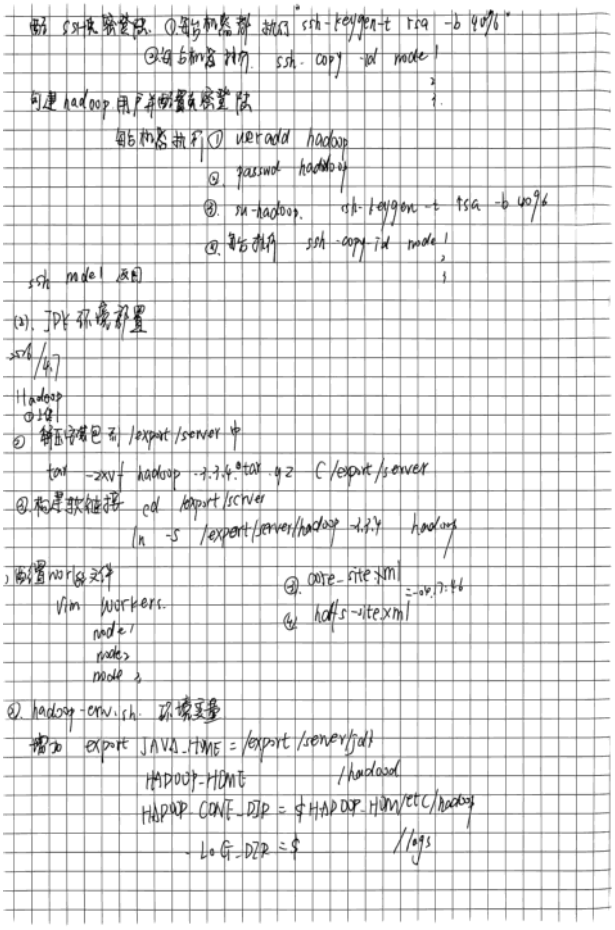
大数据分布式

数据
分布式处理技术
存 用 传
apache hadoop
处理 计算 资源调度
大数据
数字化时代
-海量数据 基础建设 生活
特征-5v 体积 种类 价值（低价值密度） 速度 质量（准确性）
核心工作：计算 存储 传输

大数据软件生态
apache hadoop
hdfs
hive
spark flink
kafka pulsar
flume sqoop
apache 软件基金会

Apache Hadoop
存储 hdfs
计算mapreduce
资源调度 yarn






HDFS

分布式的基础架构

去中心化模式

中心化模式√->主从模式 一主多从

hdfs的基础架构

hdfs集群：主角色namenode 从角色 datanode 主角色辅助 secondarynamenode

hdfs集群环境部署

node1：namenode datanode secondarynamenode

node2：datanode

node3：datanode

编辑数据目录

在model: mkdir -p /data/dn

mkdir -p /data/dn

也可和名. -p /data/dn

分发hadoop文件夹.

在中. cd /export/server

scp -r hadoop-3.3.4 node2:pwd/

相对于快捷方式

3种构建软链接

ln -s /export/server/hadoop-3.3.4 /export/server/hadoop

配置环境变量.

bin: sbin 添入 path

① vim /etc/profile

追加 export HADOOP_HOME=/export/server/hadoop

export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin

③ 先 source 让其生效执行

② source /etc/profile 保存

授权为hadoop用户

chown -R hadoop:hadoop /data.

chown -R hadoop:hadoop /export

格式化整个分布式

① 格式化namenode

su - hadoop.

hadoop namenode -format

② 启动 start-dfs.sh 在中一键启动 然后可在 http://node1:9870 查看管理网页

JPS查看正在运行的java进程

停止 stop-dfs.sh

打快照.

HDFS的shell操作

一键启停.

单独启停

hadoop-daemon.sh (start|status|stop) (namenode|secondarynode|datanode)

hdfs -daemon ( ) ( )

Linux文件系统 /

HDFS: hdfs://namenode:port/

文件系统的操作命令 hdfs dfs [ ]

1. 创建文件夹 或 hadoop fs

hdfs dfs -mkdir -p /tt/hadoop.

2. 查看指定目录下的内容 hdfs dfs -ls / [-h] [-R] [<path>]

3. 上传文件. hdfs dfs -put [-f] [-p] <localsrc> <dst>

4. 查看 hdfs dfs -cat <src> | more

5. 下载 hdfs dfs -get [-f] [-p] <src> <localsrc>

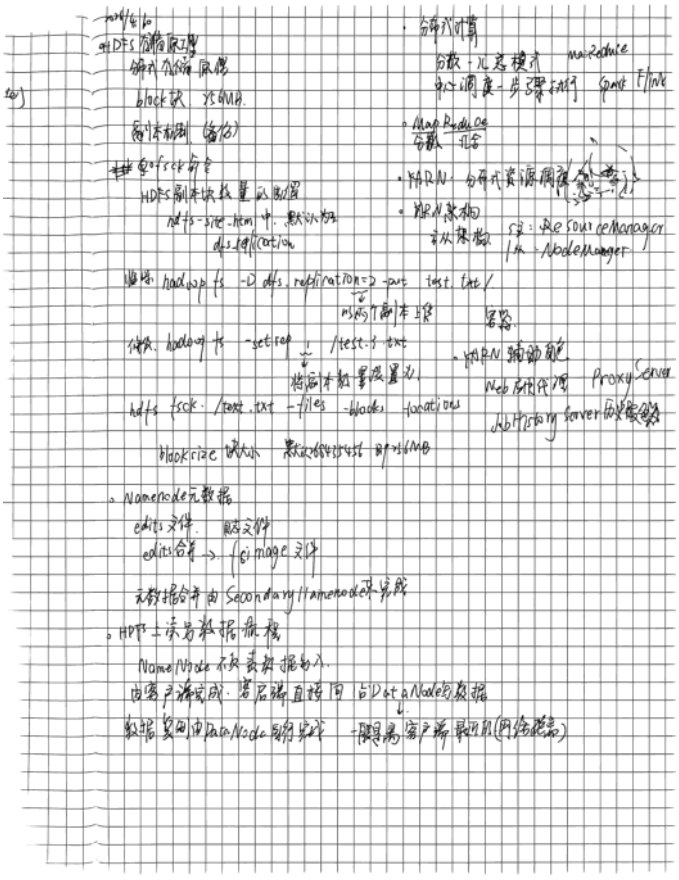
6. 拷贝 hdfs dfs -cp 覆盖 [-f] <src> <dst>

7. 追加 hdfs dfs -appendToFile <localsrc> <dst>

8. 移动 hadoop fs -mv <src> <dst>

9. 数据删除 hadoop fs -rm -r [-skipTrash] URI

删除文件夹 跳过回收站




Node1：resourcemanager nodemanager  proxyserver jobhistoryserver
Node2：nodemanager
Node3：nodemanager

mapreduce的配置

yarn的部署

日志聚合

hdfs:  node1:9870
yarn webui: node1:8088

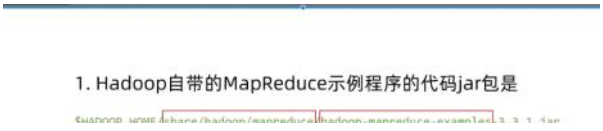
start-yarn.sh
stop-yarn.sh

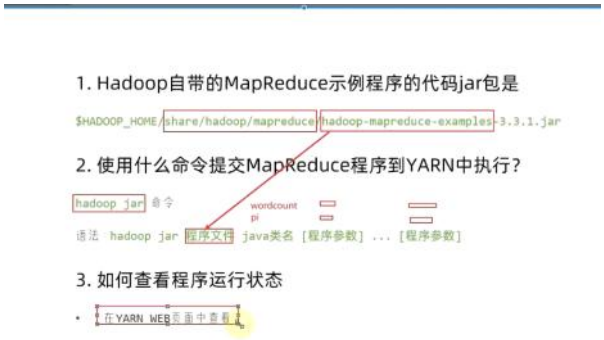
历史服务器的开关
mapred --daemon (start|stop) historyserver

单独机器的开关
yarn --daemon (start|stop) (resourcemanager|nodemanager|proxyserver)

提交mapreduce程序到yarn运行

（mapreduce内置程序：wordcount、pi）

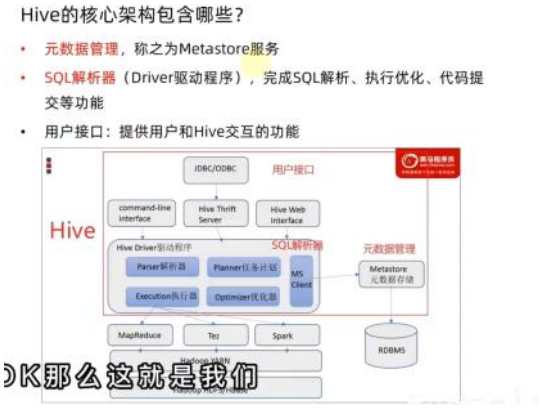





apache hive

元数据管理
sql解析器

hive架构




hive是单机工具，只需一台服务器即可，但可以提交分布式运行的mapreduce程序
tsgvR0x2Xr!f->123456

启动元数据管理服务（必须启动，否则无法工作）
前台启动：bin/hive --service metastore
后台启动：nohup bin/hive --service metastore >> logs/metastore.log 2>&1 &
启动客户端，二选一（当前先选择 Hive Shell 方式）
Hive Shell 方式（可以直接写 SQL）：bin/hiveHive
ThriftServer 方式（不可直接写 SQL，需要外部客户端链接使用）：bin/hive --service hiveserver2

```
nohup ./hive --service metastore > /tmp/metastore.log 2>&1 &
```

```
INSERT INTO test VALUES (2, '周杰伦', '男'),(3, '林俊杰', '女');
```

因为hive输出太多力所以修改了日志记录：

原始的存档：

```
rootLogger.level = ${sys:hive.log.level}
rootLogger.appenderRefs = root
rootLogger.appenderRef.root.ref = ${sys:hive.root.logger}
```

元数据记录

hive客户端

Hiveserver2

thrift端口 10000

beeline

```
!connect jdbc:hive2://node1:10000
```

Datagrip

dbeaver

创建数据库 create database if not exists myhive;

Use myhive;

Desc database myhive;

Location ->指定位置

drop database myhive；删除一个空数据库

drop database myhive cascade；强制删除连带表

数据表：

```
create table table_name(
    id int,
    name string,
    gender string
    );
```

```
su - hadoop
start-dfs.sh
start-yarn.sh
mapred --daemon start historyserver
cd /export/server/hive
nohup bin/hive --service metastore >> logs/metastore.log 2>&1 &
nohup bin/hive --service hiveserver2 >> logs/hiveserver2.log 2>&1 &
cd bin
./beeline
 !connect jdbc:hive2://node1:10000
```

输入用户名：hadoop 然后没有密码所以直接按两次回车

表：

内部表

外部表 external

1. 创建外部表语法：

- CREATE EXTERNAL TABLE .....
- 必须使用row format delimited fields terminated by指定列分隔符
- 必须使用LOCATION指定数据路径

2. 外部表和其数据，是相互独立的，即：

- 可以先有表后有数据
- 也可以先有数据，后有表
- 表和数据只是一个链接关系
- 所以删除表，表不存在了但数据保留。

查看表类型 desc  formatted 表名;

转换

- 内部表转外部表

alter table stu set tblproperties('EXTERNAL'='TRUE');

```
| Table Type:                    | EXTERNAL_TABLE
```

- 外部表转内部表

alter table stu set tblproperties('EXTERNAL'='FALSE');

```
| Table Type:                    | MANAGED_TABLE
```

通过stu set tblproperties来修改属性

要注意：('EXTERNAL'='FALSE') 或 ('EXTERNAL'='TRUE')为固定写法，区分大小写！！！

数据加载 load
load data [local] inpath '///' overwrite into table myhive.mytable;
↓
覆盖写
insert select
INSERT (OVERWRITE | INTO) TABLE tbl PARTITION(partition_key='partition_value') SELECT ... FROM ...;
数据导出insert overwrite
INSERT OVERWRITE [LOCAL] DIRECTORY  'path'  -- LOCAL可选，带LOCAL导出Linux本地，不带LOCAL导出到HDFS
[ROW FORMAT DELIMITED FIELDS TERMINATED BY ''] -- 可选，自定义列分隔符
SELECT ... FROM ...;
分区表
每一个分区是一个文件夹，文件夹之间有层级
CREATE TABLE test_ext(id int) COMMENT 'partitioned table' PARTITION BY(year string, month string, day string) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
partition分区

分桶表
分桶自动优化 set hive.enforce.bucketing=true;
创建分桶表
CREATE TABLE course (c_id string,c_name string,t_id string)
[PARTITION(partition_key='partition_value')]
CLUSTERED BY(c_id) INTO 3 BUCKETS ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';

分桶表加载数据只能用insert select
一般是先创建一个临时表，在临时表中loaddata，然后再通过insert select映射到分桶表

```
INSERT (OVERWRITE | INTO) TABLE tbl
[PARTITION(partition_key='partition_value')]
SELECT ... FROM ... CLUSTER BY(col);
```

修改表
属性

```
ALTER TABLE tbl SET TBLPROPERTIES(key=value);
-- 常用属性
("EXTERNAL"="TRUE") -- 内外部表，TRUE表示外部表
('comment' = new_comment) -- 修改表注释
```

-- 其余属性参见
https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL#LanguageManualDDL-listTableProperties

添加分区
ALTER TABLE tablename ADD PARTITION (partition_key='partition_value', ......);
修改分区值->修改元数据记录，但是hdfs的实体文件夹不会改名
ALTER TABLE tablename PARTITION (partition_key='old_partition_value') RENAME TO PARTITION (partition_key='new_partition_value');
删除分区
alter table score drop partition(year='2019',month='10',day='07')
添加新列
alter table score add columns(v1 int,v2 string);
修改列名
alter table score change v2 v2new string;
删除表
drop table myhive.score;
清除数据表->内部表可以外部表不可以
truncate table course;

array类型
array<string> array<int>
create table myhive.test(work_locations arry<string>)
row format delimited fields terminated by '\t'
collection item terminated by '///';
↓
数据中的分隔符， | 等等什么的
select work_locations[0] from myhive.test;
array_contains()数组是否包含

map类型
key-value键值对类型
map<string,string>

```
create table myhive.test_map(
id int, name string, members map<string,string>, age int
)
row format delimited
fields terminated by ','
COLLECTION ITEMS TERMINATED BY '#'
MAP KEYS TERMINATED BY ':';
```

列分隔符 row format
键值对之间的分隔
键值对内的分隔
取key map_key()
取value map_value()
map个数：select size(members) form myhive.text
array_contains(map_keys(members),'sister') 查看是否包含

struct结构 作为一个整体的列
struct<name:string,age:int,createtime:data>

```
create table myhive.test_struct(
id string, info struct<name:string, age:int>
)
row format delimited
fields terminated by '#'
COLLECTION ITEMS TERMINATED BY ':';
```

基本查询

```
SELECT [ALL | DISTINCT]select_expr, select_expr, ...
FROM table_reference
[WHERE where_condition]
[GROUP BYcol_list]
[HAVING where_condition]
[ORDER BYcol_list]
[CLUSTER BYcol_list
  | [DISTRIBUTE BY col_list] [SORT BY col_list]
]
[LIMIT number]
```

where useraddress like '%广东%'；模糊匹配
order by
limit 1—>第一个

分组、聚合
对聚合的结果做判断用having
join

正则匹配 rlike '.*广东.*'
. 任意字符 *任意数量

[abc].* abc任意字符开头

+一次或多次 *任意

\s 空白字符 \S非空白字符

转义字符

union联合

select ... and select

默认去重

union all 不去重

随机采样

tablesample

随机桶抽样

select ... from tbl tablesample(bucket x out of y on(colname | rand())) 列的哈希桶 或者完全随机

基于数据块抽样 select ... from tbl tablesample(name rows | num percent | num(K|M|G))　　kmg分别为kb mb gb

虚拟列

row_ _offset_ _inside_ _block记得先改为true : set hive.exec.rowoffset=true;

(两个虚线，这里中间多加了空格为了看清楚ovo)

函数

select

round() 取整

rand()随机数

abs()绝对值

pi()

size()元素个数

map_keys()

map_values()

array_contains()

sort_array()

binary()转化为二进制

cast()自由转化

current_timestamp()当前时间

to_data()时间戳转日期
year()取年份 month() quarter()季度 dayofmonth()天 hour() minute() second() weekofyear() datediff()日期之间的天数
date_add() date_sub()日期相减

条件函数
if（true is null,'不知道'，truename）from users；
isnull()null判断 isnotnull（）
nvl()
coalesce()
case ... when ... then ... when ... then ... else ... end from ...

trim()去除首位字符
hash()哈希加密